

THALUWA BAZAR — React Native MVP Specification

Production-Ready Mobile Architecture for Assam’s Hyperlocal Marketplace Startup

1. Startup Vision & Mobile Product Scope

Thaluwa Bazar is transitioning from a web proof-of-concept to a **native mobile MVP** built on React Native & Expo. The platform connects rural and semi-urban producers (farmers, fishermen, handloom artisans, home cooks) directly with local urban buyers within a configurable radius (1–25 km). This specification outlines the complete architecture, navigation, secure state machine, and offline-resilient mobile engineering required for production launch.

Buyer Mobile Experience

- Location-first onboarding & GPS radius filter (1km–25km).
- Bilingual UI (Assamese & English) with instant toggle.
- Direct order placement with trusted server price calculations.
- Secure phone number unlock & in-app order tracking.

Seller & Admin Apps

- Real-time push notifications for incoming orders (Firebase/Expo Push).
- Order state progression: REQUESTED → ACCEPTED → READY.
- Inventory management, photo uploads, & verification status tracking.
- Moderation dashboard & platform analytics for founders.

2. React Native & Mobile Tech Stack

Layer	Technology & Libraries
Framework	React Native (via Expo SDK 51+ managed workflow with EAS Build) for cross-platform iOS & Android deployment.
Navigation	Expo Router (File-based native routing mirroring app directory structures).
Styling	NativeWind (Tailwind CSS for React Native) ensuring design token consistency.
State & Cache	Zustand (client UI state), TanStack Query (React Query) for server caching, optimistic updates, and synchronization.
Backend & DB	Supabase (PostgreSQL with PostGIS extension for spatial queries, Auth, Edge Functions, & Realtime WebSockets).
Native Modules	Expo Location (GPS), Expo Notifications (Push), AsyncStorage (Local secure cache).

Startup Directive: Never trust client-side calculations for inventory, distance formulas, or totals. All state transitions and fees are enforced via Supabase Row Level Security (RLS) and PostgreSQL Database Transactions.

3. Production Database Schema (PostgreSQL / Supabase)

The schema enforces strict referential integrity, spatial indexing for hyperlocal matching, and append-only auditing.

Table Name	Core Fields & Purpose
users	id, phone, email, role (buyer/seller/moderator/admin), language, lat, lng, verified, created_at
seller_profiles	user_id, business_name, UPI/bank details, verification_status, rating, delivery_radius_km
listings	id, seller_id, category_id, title_as, title_en, description, price, unit, stock, geom (PostGIS point), status
orders	id, buyer_id, seller_id, subtotal, delivery_fee, total, status (state machine), pickup_or_delivery, created_at
order_items	id, order_id, listing_id, quantity, unit_price, line_total
contact_unlocks	id, buyer_id, seller_id, listing_id, payment_status, created_at (Privacy enforcement log)
notifications	id, user_id, type, title, body, data_json, read_at, push_sent_status, created_at

4. Critical Backend Workflow: Order Placement

When a buyer submits an order from the mobile app, the backend handles the transaction atomically:

- **1. Auth & Validation:** Supabase Auth verifies mobile session token; backend checks listing stock and active status.
- **2. Transaction Lock:** PostgreSQL executes a transaction: decrements stock, calculates trusted totals on server, and inserts record into orders and order_items.
- **3. Audit & Notification:** Writes an entry to audit_logs and triggers an asynchronous notification row.
- **4. Push Delivery:** Supabase Edge Function dispatches an Expo Push Notification to the seller's mobile device.

5. Order State Machine & Real-Time Sync

Allowed state transitions are strictly governed by backend database triggers and API route guards:



Supabase Realtime subscribes the seller's React Native app to the orders table. Upon receiving a new order row, the app triggers an in-app sound, haptic vibration (via expo-haptics), and updates the dashboard instantly without manual refresh.

6. Mobile Navigation Structure (Expo Router)

Public & Buyer Tabs

- `/(auth)/login`: OTP phone verification via Supabase.
- `/(tabs)/explore`: Hyperlocal feed with radius slider.
- `/(tabs)/haats`: Traditional Assam market hubs view.
- `/(tabs)/orders`: Live tracking & order history.
- `/(tabs)/profile`: Language switch (Assamese/English).

Seller & Admin Stacks

- `/seller/dashboard`: Incoming orders & live toggle.
- `/seller/inventory`: Add/edit product with camera upload.
- `/moderator/reports`: Flagged items review.
- `/admin/economics`: Platform commission & audit logs.

7. Hyperlocal Engine & PostGIS Integration

Unlike hackathon memory filters, the MVP utilizes PostgreSQL with **PostGIS** geography types. Listings are queried efficiently using spatial indexes:

```
SELECT * FROM listings WHERE ST_DWithin(geom, ST_MakePoint(lng, lat)::geography, radius_meters) AND status = 'active';
```

8. Seller Contact Privacy Protection

Direct phone numbers are hidden in public listing payloads. Buyers must tap "Unlock Contact", generating a secure audit log in `contact_unlocks` and a micro-transaction record before the server exposes the masked or direct phone endpoint.

9. Step-by-Step Mobile MVP Execution Plan

- **Phase 1**: Initialize Expo SDK project with NativeWind, TypeScript, and Expo Router directory layout.
- **Phase 2**: Provision Supabase database, run migration scripts for schema, RLS policies, and PostGIS setup.
- **Phase 3**: Implement Supabase Auth (Phone OTP) and multilingual context (Assamese/English).
- **Phase 4**: Build Explore Feed with expo-location and PostGIS distance filters.
- **Phase 5**: Implement order transaction flow, state machine, and Expo Push notifications.
- **Phase 6**: Build Seller Dashboard with real-time Supabase subscriptions and camera listing uploads.
- **Phase 7**: Perform rigorous security testing (OWASP mobile baseline) and build internal TestFlight / APK distribution.